

머신러닝 7-1 교시

부스팅 boosting

학습 목표

- 부스팅이 배깅과 본질적으로 다른 "순차" 학습 방식임을 설명할 수 있다.
- 그래디언트 부스팅이 "잔차(residual) 를 새 모델로 예측한다" 는 발상에서 출발했음을 설명할 수 있다.
- XGBoost · LightGBM · CatBoost 의 사이킷런 호환 API 로 분류·회귀 모델을 학습할 수 있다.
- `learning_rate` · `n_estimators` · `max_depth` · `num_leaves` · `subsample` 의 상호작용을 안다.
- 클래스 불균형 데이터에서 `class_weight`, `scale_pos_weight`, SMOTE 를 적용할 수 있다.

4 장. 부스팅의 아이디어

여기까지는 "여러 모델을 병렬로 학습한 뒤 평균" 이었다. 이번에는 정반대 발상을 본다 — "하나씩 차례로 만들어 가되, 다음 모델이 앞 모델의 약점을 메우게 한다." 부스팅(Boosting) 이다. "끌어올리다" 라는 영어 단어가 그대로 이름이 됐다.

4.1 직관 — 다음 모델이 앞 모델의 약점을 메운다

모든 부스팅 알고리즘의 공통 골격은 다섯 단계다.

부스팅의 일반적 흐름

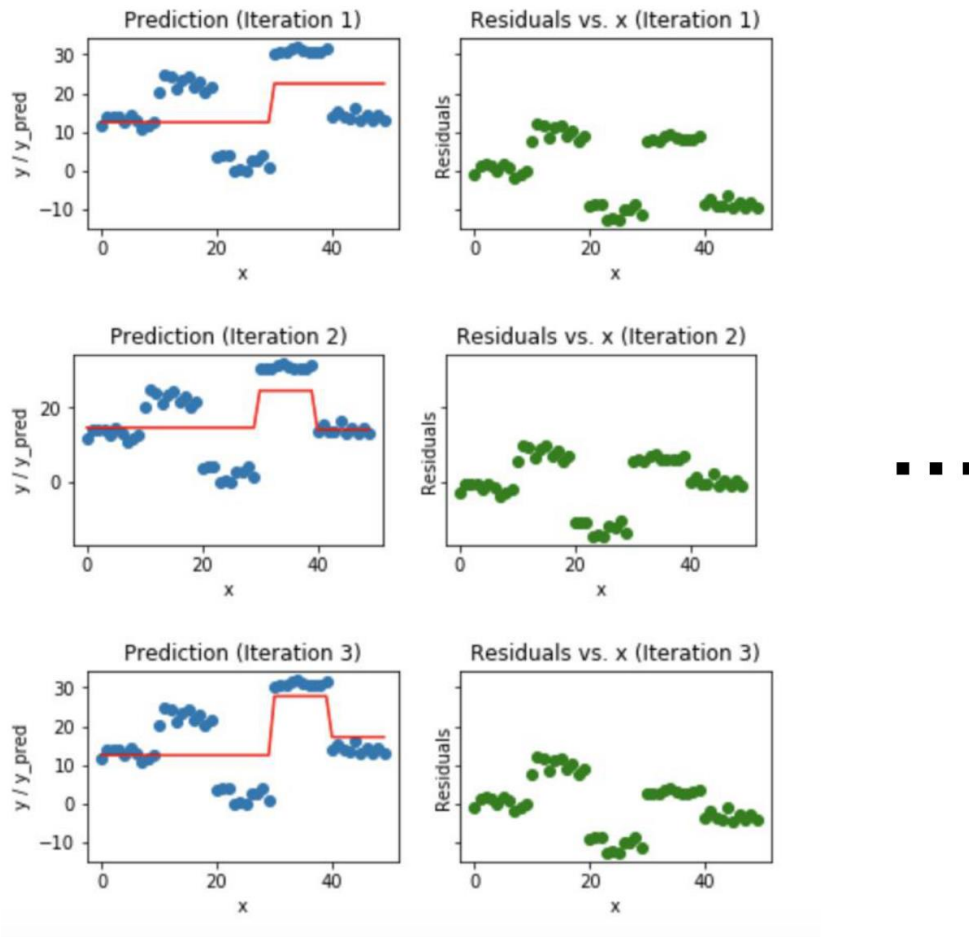
- 1) 약한 학습기(weak learner) m_1 을 학습한다 (보통 얇은 트리)
- 2) 그 결과의 "오류"를 본다
- 3) 그 오류를 줄이도록 다음 학습기 m_2 를 학습한다
- 4) 위 과정을 N 번 반복
- 5) 모든 학습기의 예측을 가중 합산

약한 학습기

- 약한 학습기(weak learner) – 성능이 무작위 추정보다 "조금 나은" 모델. 부스팅에서는 보통 깊이 1~5 인 매우 얇은 결정트리를 쓴다.
- 단독으로는 형편없지만, 여러 개가 순차로 누적되면 매우 강력해진다 – "단순한 것들이 함께 일하면 강해진다" 는 부스팅 철학.

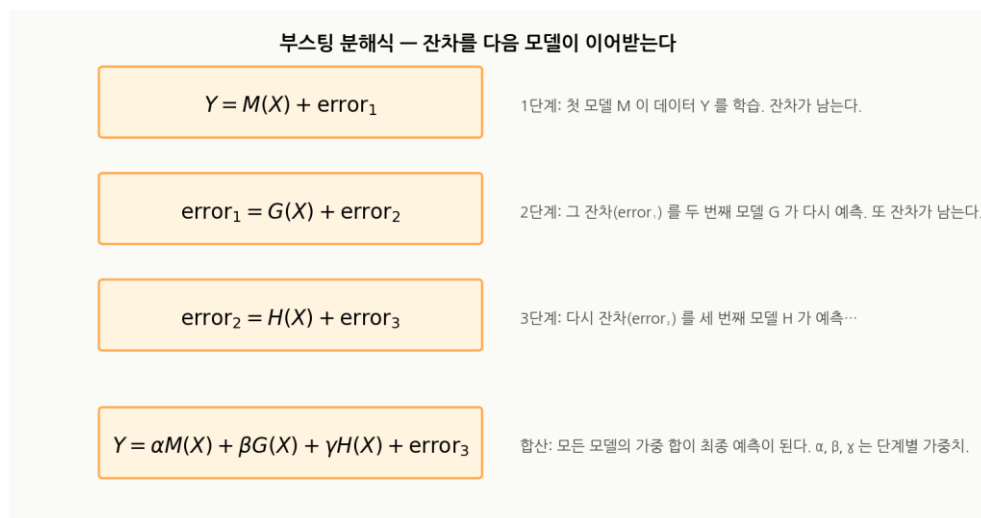
4.2 부스팅의 분해식

슬라이드의 분해식이 부스팅의 정수를 잘 보여 준다. 예측을 단계별로 "풀어 쓰면" 부스팅이 무엇을 하는지가 한눈에 들어온다.



source: <https://medium.com/mlreview/gradient-boosting-from-scratch-1e317ae4587d>

[그림 4-1] 부스팅 분해식 — $Y = \alpha M(X) + \beta G(X) + \gamma H(X) + \text{error}_3$. — 출처: 강의자료 p.68.



[그림 4-2] 부스팅 분해식 — 잔차를 다음 모델이 이어받는다. α, β, γ 는 단계별 가중치.

마지막 줄을 다시 보자 — $Y \approx \alpha M(X) + \beta G(X) + \gamma H(X)$. 이게 부스팅 최종 예측 식이다. 가중치 α, β, γ 는 알고리즘마다 정해지는 방식이 다르다, 그래디언트 부스팅은 learning_rate 와 앞노드 값으로 결정된다.

4.3 배깅과의 본질 차이 — 한눈 표

배깅(랜덤포레스트) vs 부스팅

└─ 학습 방식 : 병렬 vs 순차

└─ 약한 학습기 : 보통 깊은 트리 vs 얇은 트리

└─ 분산/편향 : 분산 감소 vs 편향 감소

└─ 과적합 위험 : 낮음 vs 중간 (튜닝 중요)

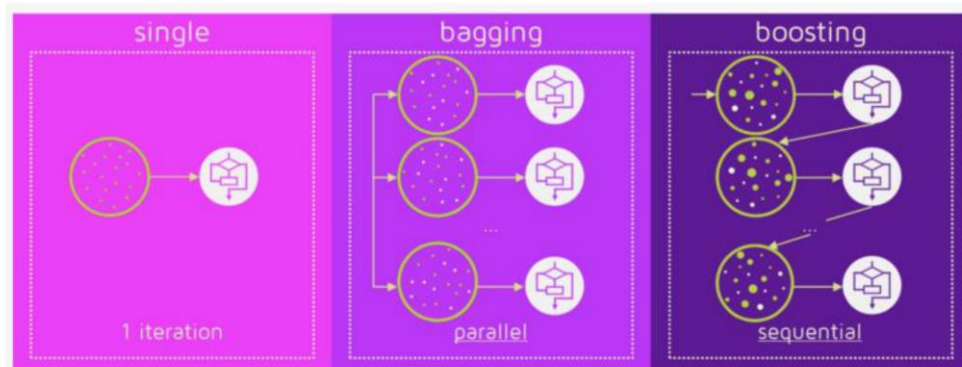
└─ 학습 시간 : 트리 개수에 비례 (병렬 가능)

└─ | vs n_estimators 에 비례 (순차)

└─ 추천 상황 : "안정적 베이스라인"이 필요할 때 vs 정밀도 끝까지 올리고 싶을 때 (대회·실무 1 순위)

축	배깅 (랜덤포레스트)	부스팅
학습 방식	병렬 (각 트리 독립)	순차 (앞 모델 결과를 다음에 전달)
기본 학습기	깊은 트리 (편향 ↓)	얇은 트리 (분산 ↓)
줄여주는 것	분산	편향
과적합 위험	낮음	중간 — 튜닝이 중요
병렬화	쉬움 (n_jobs=-1)	원리상 어려움 (트리 내부는 가능)
하이퍼파라미터	n_estimators, max_features 가 핵심	learning_rate × n_estimators 가 핵심
대표 모델	RandomForest, ExtraTrees	AdaBoost, GBM, XGBoost, LightGBM, CatBoost
추천 상황	안정적 베이스라인	정밀도 끝까지 짜내기

- 단일 알고리즘, 배깅, 부스팅의 차이



source: <https://medium.com/mireview/gradient-boosting-from-scratch-1e317ae4587d>

[그림 4-3] 단일 알고리즘 · 배깅 · 부스팅 한눈 비교.

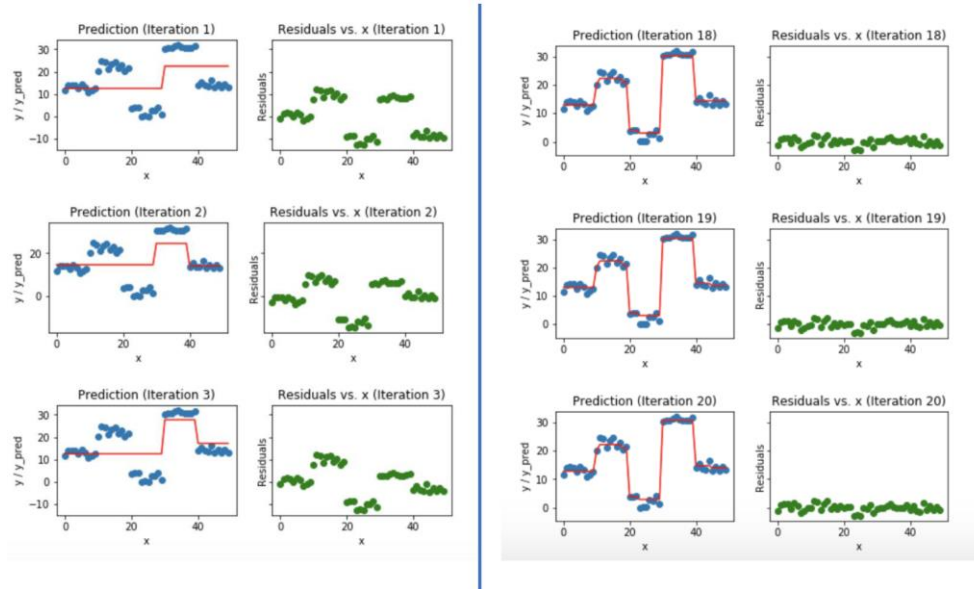
5 장. 그래디언트 부스팅(Gradient Boosting)

그래디언트 부스팅은 부스팅을 "잔차 학습"의 관점으로 다시 정리한 알고리즘이다. AdaBoost가 "가중치 분배"였다면, 그래디언트 부스팅은 "잔차(residual) 학습"이라고 부를 수 있다.

1999년 제롬 프리드만(Jerome Friedman)이 처음 정리했다 — 랜덤 포레스트의 브레이만과 같은 스탠퍼드 학파다.

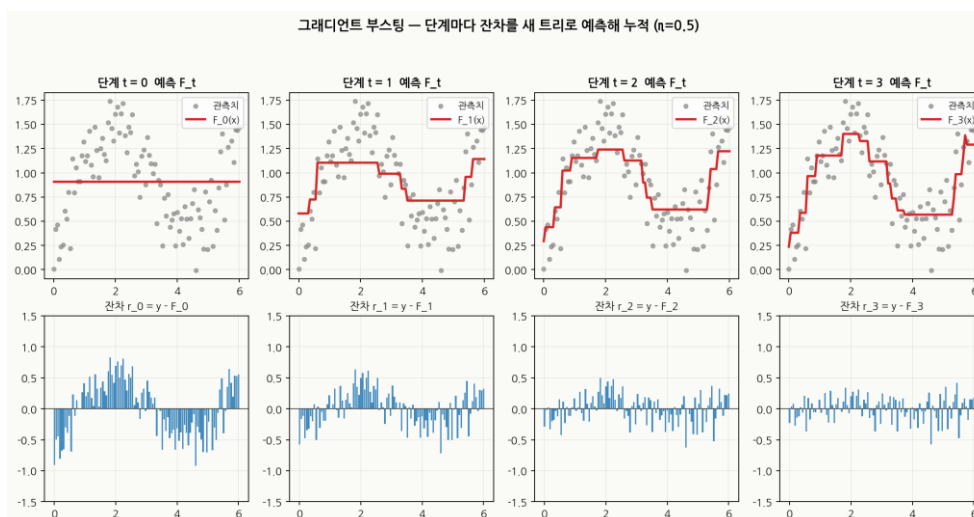
5.1 잔차 학습이라는 발상

회귀를 예로 든다. 단순히 평균 $\bar{y}$ 을 예측한다고 하자. 그러면 각 데이터에 대해 잔차 $(y - \bar{y})$ 가 남는다. 이 잔차를 새 모델이 다시 예측한다. 그 결과를 더하면 예측이 살짝 정교해진다. 또 잔차가 남는다. 또 새 모델로 학습한다. ... 이를 N 번 반복한다.



source: <https://medium.com/mlreview/gradient-boosting-from-scratch-1e317ae4587d>

[그림 6-1] 부스팅 분해식의 가중치 학습 — $Y = \alpha M(X) + \beta G(X) + \gamma H(X)$.



[그림 6-2] 그래디언트 부스팅 — 단계마다 잔차를 새 트리로 예측해 누적한다 ($\eta=0.5$).

회귀 손실(MSE) 기준 GBM

초기 예측 $F_0(x) = \bar{y}$ (그냥 전체 평균)

반복 $t = 1, \dots, T$:

- 1) 잔차 $r_i = y_i - F_{t-1}(x_i)$ (지금까지의 예측 오차)
- 2) 새 학습기 h_t 가 잔차 r 을 예측하도록 학습
- 3) 예측 갱신: $F_t(x) = F_{t-1}(x) + \eta \times h_t(x)$
(η : learning_rate, 보통 0.05~0.3)

최종 예측 = $F_T(x)$

6.1.1 "음의 그라디언트 = 일반화된 잔차"

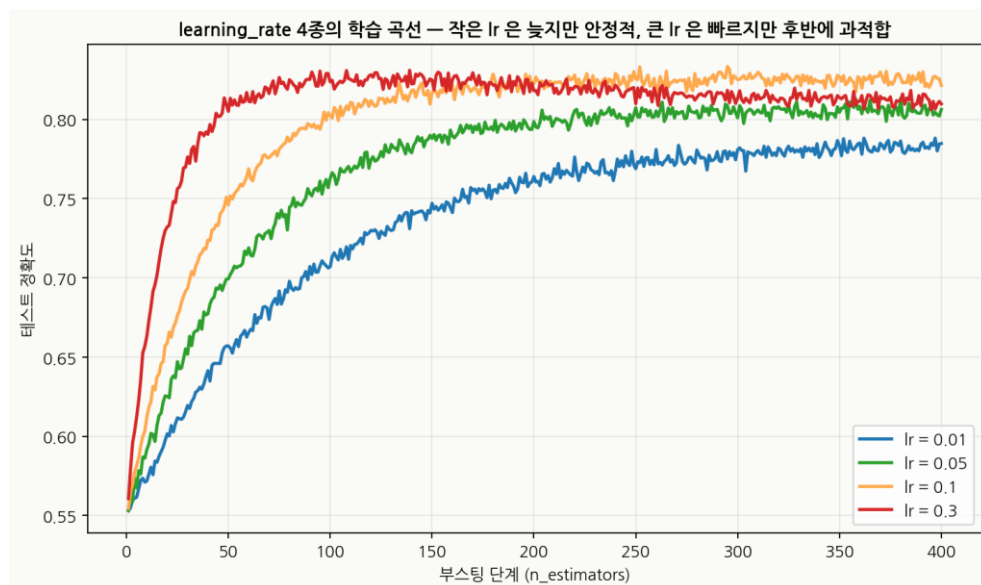
MSE 손실의 음의 그라디언트는 정확히 $(y - F(x))$ — 즉 잔차다. 이 사실을 일반화해 다른 손실 함수에도 동일한 골격을 적용할 수 있다. 분류의 로그 손실, 분위 회귀의 quantile loss, 견고 회귀의 Huber loss — 모두 음의 그라디언트를 "잔차처럼" 사용해 같은 부스팅 식을 적용한다. 그래서 "그라디언트" 부스팅이다.

6.2 핵심 하이퍼파라미터의 트레이드오프

부스팅 모델 핵심 파라미터

- └ learning_rate (η) : 한 번에 얼마만큼 더할지 (보통 0.01~0.3)
 - └ 작을수록 안정적이나, n_estimators 를 늘려야 함
- └ n_estimators : 트리(부스팅 단계) 개수
 - └ learning_rate 와 곱이 "총 학습량"이라 생각
- └ max_depth : 개별 트리 깊이 (3~8 흔히)
 - └ 부스팅의 트리는 약한 학습기 - 너무 깊으면 과적합
- └ subsample : 각 단계에 사용할 샘플 비율 (Stochastic GBM)
- └ min_child_weight / min_samples_leaf : 잎 노드 최소 샘플 수

실무 팁 — learning_rate 를 작게(예 0.05) 잡고 n_estimators 를 충분히 크게 두면, 일반적으로 더 좋은 모델이 나온다. 다만 학습 시간이 길어진다.



[그림 6-3] learning_rate 4 종의 학습 곡선 — 작은 lr 은 느리지만 안정적, 큰 lr 은 빠르지만 후반에 과적합.

6.3 손실 함수의 다양성

분류/회귀	손실 함수	특징
회귀	MSE (squared)	기본값. 이상치에 민감.
회귀	MAE (absolute)	이상치 견고. 분포 중앙값 추정.
회귀	Huber	MSE+MAE 절충. 이상치 견고하면서 미분가능.

회귀	Quantile	분위수 예측 (예 — 95% 신뢰 구간 상한).
분류	Log-Loss (deviance)	기본값. 로지스틱 회귀와 같은 손실.
분류	Exponential	AdaBoost 와 동일한 손실.

6.4 코드 실습 — GradientBoostingClassifier [2 단계]

```
gbm = GradientBoostingClassifier(
    n_estimators=300, learning_rate=0.05, max_depth=3,
    random_state=42)
gbm.fit(X_tr, y_tr)
print("GBM 정확도:", gbm.score(X_te, y_te))
```

사이킷런 기본 GradientBoostingClassifier 는 학습이 느리다(순수 파이썬 구현). Titanic 같은 작은 데이터에서는 충분하지만, 큰 데이터에서는 다음 장의 XGBoost / LightGBM 을 쓴다.

6.5 6 장 함정/팁

그래디언트 부스팅 함정/팁

- learning_rate 와 n_estimators 는 "역수 관계" 다. 한쪽을 줄이면 다른 쪽을 늘려 보상.
- max_depth 는 3~8 사이가 무난. 부스팅에서 깊은 트리는 거의 항상 과적합으로 가는 길.
- early_stopping 을 적극 활용해 "검증 점수가 더 이상 안 오르면 멈춤" 으로 시간 절약 + 과적합 방지.

7 장. 현대 부스팅 라이브러리 — XGBoost · LightGBM · CatBoost

"단순 그래디언트 부스팅" 을 산업 규모로 효율화한 라이브러리들이 등장하면서 부스팅은 캐글 (Kaggle) 대회와 실무의 표준이 되었다. 이 장은 본 책의 핵심 강화 부분이다 — 특히 XGBoost 와 LightGBM 은 실전 예제와 함께 더 자세히 다룬다.

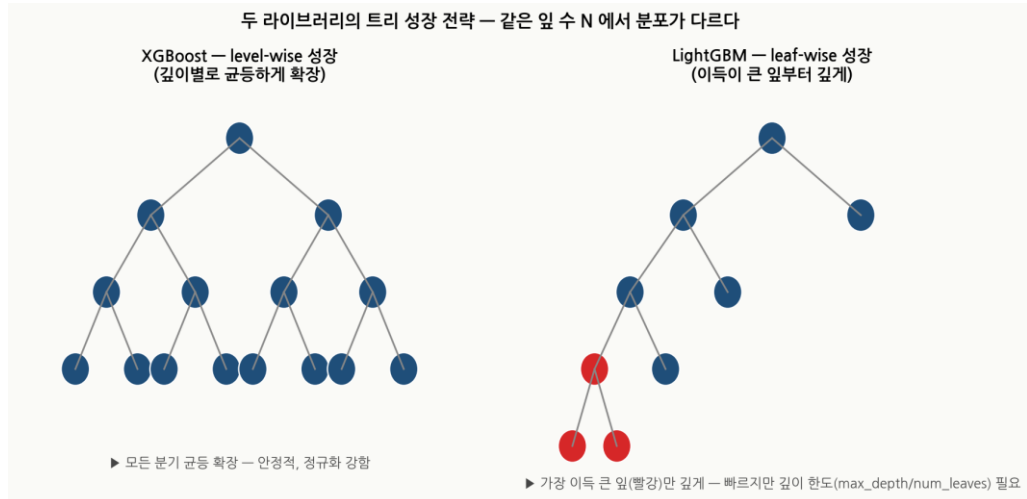
7.1 왜 새로운 구현이 필요했는가

- 데이터가 커지면서 사이킷런 기본 GBM 의 "순수 파이썬 + 단일 코어" 구현으로는 학습 시간이 폭증했다.
- 정규화(L1/L2) 와 가지치기(pruning) 같은 안전장치가 기본으로 들어 있지 않아 과적합이 쉬웠다.
- 범주형 변수와 결측치 처리가 번거로웠다 — one-hot 인코딩과 imputation 을 매번 손으로 해야 했다.
- 이런 한계를 정면 돌파한 것이 2014 년의 XGBoost, 2016 년의 LightGBM, 2017 년의 CatBoost 다.

현대 부스팅 라이브러리 비교

└─ XGBoost : 정밀도, 정규화(L1/L2)가 강함. 속도 빠름.
| └─ 트리 성장 방식: level-wise (깊이별)

- └ 사이킷런 호환 API: XGBClassifier, XGBRegressor
- └ LightGBM : 보통 가장 빠름. 큰 데이터에 강함.
 - └ 트리 성장 방식: leaf-wise (가장 큰 잎부터)
- └ 사이킷런 호환 API: LGBMClassifier, LGBMRegressor
- └ CatBoost : 범주형 변수 처리가 자연스러움. 기본값이 좋음.
 - └ 순서형 부스팅(target leakage 방지)
 - └ 사이킷런 호환 API: CatBoostClassifier



[그림 7-1] XGBoost level-wise 성장 vs LightGBM leaf-wise 성장 — 같은 잎 수에서 분포가 다르다.

7.2 XGBoost

7.2.1 알고리즘 핵심 원리

XGBoost(Extreme Gradient Boosting, Tianqi Chen, 2014)의 차별점은 두 가지다. 첫째, 손실 함수를 "2차 테일러 전개"까지 근사해 잎노드 값을 "닫힌 형태"로 한 번에 계산한다. 사이킷런 기본 GBM이 1차(gradient)만 쓰는 데 비해, XGBoost는 2차(hessian) 정보까지 활용해 더 좋은 분할을 찾는다. 둘째, 트리 자체를 정규화한다 — 잎노드 개수에 γ 페널티, 잎 값에 λ L2 페널티를 부과해 과적합을 강하게 억제한다.

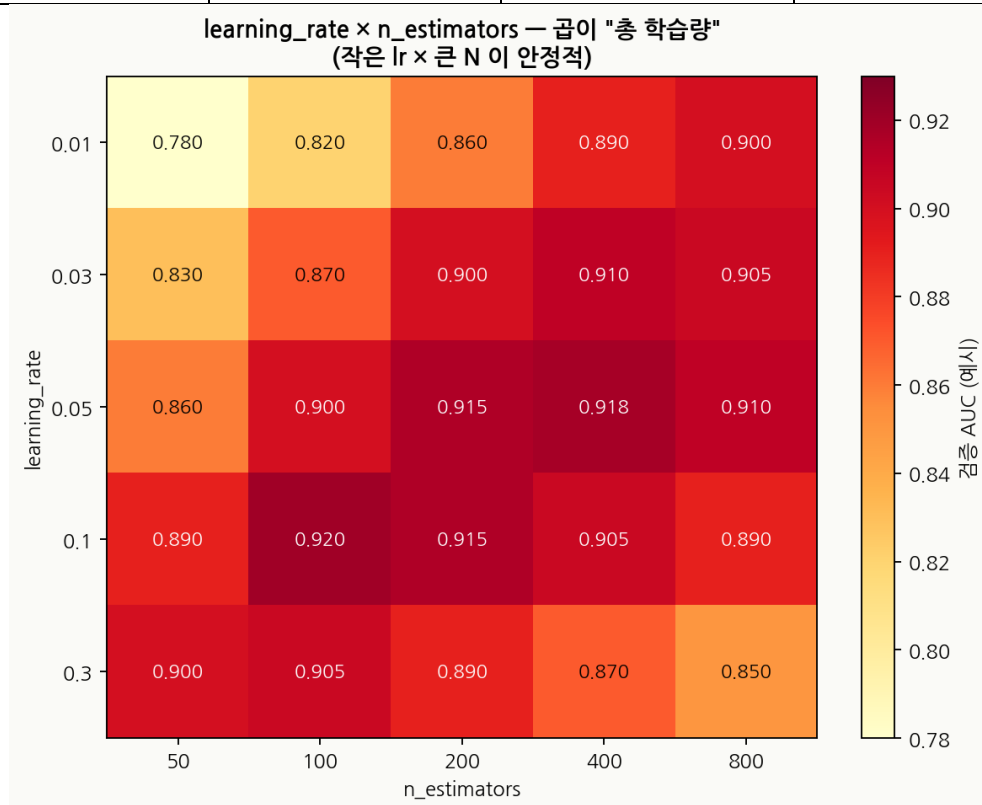
XGBoost가 다른 점

- Level-wise 성장 — 깊이별로 모든 잎을 균등하게 확장. 안정적이지만 LightGBM보다 느리다.
- 정규화 목적함수 — $L(\theta) = \sum \text{Loss} + \gamma T + (1/2)\lambda \sum w^2$ 형태. T 는 잎 수, w 는 잎 값.
- 결측치 자동 처리 — 학습 중 결측치를 어느 방향으로 보내는 게 좋은지 자동으로 결정.

7.2.2 주요 하이퍼파라미터 상세 표

파라미터	의미	권장 범위	튜닝 우선순위
learning_rate (eta)	한 단계 학습량	0.01 ~ 0.3	1
n_estimators	트리 개수 (부스팅 단계)	100 ~ 1000+	1
max_depth	개별 트리 깊이	3 ~ 10	2

min_child_weight	잎 노드 최소 가중치 합	1 ~ 10	2
subsample	각 단계 샘플 비율	0.7 ~ 1.0	3
colsample_bytree	각 트리에 사용할 특 성 비율	0.7 ~ 1.0	3
gamma (min_split _loss)	분할 최소 손실 감소	0 ~ 5	3
lambda (reg_lamb da)	요 값 L2 페널티	0 ~ 5	4
alpha (reg_alpha)	요 값 L1 페널티	0 ~ 1	4
scale_pos_weight	양성 클래스 가중치 (불균형)	neg/pos	1 (불균형 시)
early_stopping_ro unds	검증 점수 정체 시 중 단	20 ~ 100	2
tree_method	분할 알고리즘	hist (기본), gpu_his t	5 (속도)



[그림 7-2] learning_rate × n_estimators 의 곱이 "총 학습량" — 작은 lr × 큰 N 이 안정적인 영역이다.

7.2.3 실전 예제 ① — 이진 분류 + early_stopping (Titanic) [3 단계]

```
# 사전 설치: pip install xgboost
from xgboost import XGBClassifier
```

```
xgb = XGBClassifier(
    n_estimators=500, learning_rate=0.05, max_depth=4,
    subsample=0.8, colsample_bytree=0.8,
    eval_metric="logloss", random_state=42, n_jobs=-1)
xgb.fit(X_tr, y_tr)
print("XGBoost 정확도:", xgb.score(X_te, y_te))
```

더 실전적인 형태로 early_stopping 까지 적용해 보자.

```
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split

# 학습용을 다시 train / val 로 분리 (검증용)
X_train, X_val, y_train, y_val = train_test_split(
    X_tr, y_tr, test_size=0.2, stratify=y_tr, random_state=42)

xgb = XGBClassifier(
    n_estimators=1500,                # 일부러 크게 잡고 early_stopping 에 맡김
    learning_rate=0.03,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    eval_metric="auc",
    early_stopping_rounds=50,
    random_state=42, n_jobs=-1)

xgb.fit(X_train, y_train,
        eval_set=[(X_val, y_val)],
        verbose=False)

print("최적 트리 수:", xgb.best_iteration)
print("검증 AUC   :", xgb.best_score)
print("테스트 정확도:", xgb.score(X_te, y_te))
```

early_stopping 은 검증 AUC 가 50 라운드 동안 개선되지 않으면 학습을 중단한다.

n_estimators 를 1500 으로 크게 잡아도 실제 학습은 best_iteration 까지만 진행되므로 안전하다. Titanic 에서는 보통 best_iteration 이 200~400 사이로 잡힌다.

7.2.4 실전 예제 ② — 다중 분류 (Iris)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier

iris = load_iris()
X, y = iris.data, iris.target
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

xgb = XGBClassifier(
    objective="multi:softprob",      # 다중 분류 (확률 출력)
    num_class=3,
    n_estimators=200,
    learning_rate=0.1,
    max_depth=4,
```

```

    random_state=42, n_jobs=-1)
xgb.fit(X_tr, y_tr)

print("Iris 다중 분류 정확도:", xgb.score(X_te, y_te))

```

objective="multi:softprob" 는 클래스별 확률을 반환한다. 단순 라벨만 필요하면 "multi:soft max" 를 쓴다. num_class 를 명시해야 한다.

7.2.5 실전 예제 ③ — 회귀 + 특성 중요도 (California Housing)

```

from sklearn.datasets import fetch_california_housing
from xgboost import XGBRegressor
import matplotlib.pyplot as plt
import pandas as pd

data = fetch_california_housing()
X, y = pd.DataFrame(data.data, columns=data.feature_names), data.target

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

xgb_r = XGBRegressor(
    n_estimators=600, learning_rate=0.05, max_depth=6,
    subsample=0.85, colsample_bytree=0.9, reg_lambda=1.0,
    random_state=42, n_jobs=-1)
xgb_r.fit(X_tr, y_tr)

print("R²:", xgb_r.score(X_te, y_te))

# 특성 중요도 추출
imp = pd.Series(xgb_r.feature_importances_, index=X.columns).sort_values()
imp.plot(kind="barh", title="XGBoost 회귀 - 특성 중요도", figsize=(8, 5))
plt.tight_layout(); plt.show()

```

California Housing 에서는 $R^2 \approx 0.84$ 정도가 무난히 나온다. 랜덤포레스트(0.81) 보다 살짝 우위. 특성 중요도 plot 으로 MedInc(중위 소득) 와 Latitude/Longitude(지리적 위치) 가 가장 영향이 큰 것이 보인다.

7.2.6 XGBoost 흔한 함정과 팁

XGBoost - 흔한 함정과 팁

- 검증 세트 분리를 빼먹지 말 것. **early_stopping** 은 검증 세트가 있어야 작동한다.
- n_estimators 를 크게 잡고 early_stopping 에 맡기는 패턴이 가장 안전하다.
- reg_lambda, reg_alpha 같은 정규화는 "먼저 max_depth·learning_rate 로 모델 안정시킨 뒤" 손대는 게 좋다.
- 결측치를 그대로 받지만, 무엇이 결측인지 일관된 표기(NaN) 가 중요하다. 0 으로 채운 "가짜 결측" 은 알아채지 못한다.
- tree_method="hist" 는 LightGBM 의 히스토그램 기반과 비슷한 속도를 제공한다 - 큰 데이터에서 권장.
- scale_pos_weight = neg/pos 는 "매우 작은 양성 비율" 일 때만 의미. 비율이 1:3 이내라면 효과가 미미.

7.3 LightGBM — leaf-wise 성장과 히스토그램 기반

7.3.1 알고리즘 핵심 원리

LightGBM(Microsoft, 2016) 의 차별점은 세 가지다. 첫째, leaf-wise 성장 — 깊이별이 아니라 "이득이 가장 큰 잎" 부터 깊게 키운다. 같은 잎 수에서 더 깊은 트리가 되므로 학습이 빠르고 정확도가 높은 편이지만, 깊이 한도(max_depth 또는 num_leaves) 가 없으면 과적합으로 직행한다. 둘째, 히스토그램 기반 분할 — 연속값을 256 개 구간으로 미리 잘라 두고 그 구간 단위로만 분할 후보를 본다. 정확도 손실은 거의 없으면서 속도가 매우 빠르다. 셋째, 큰 데이터·고차원 희소 데이터에서 더 빠르다.

LightGBM 이 빠른 이유

- Leaf-wise 성장 – 가장 손실을 많이 줄일 "잎" 한 개만 골라 분할. 같은 트리 수에서 더 정밀하지만 과적합 위험 ↑.
- 히스토그램 기반 – 연속값 → 256 개 bin. 메모리·속도 모두 유리.
- GOSS – 큰 gradient 의 샘플을 우선 유지, 작은 gradient 의 샘플은 일부만 무작위 추출.
- EFB – 서로 동시에 0 이 아닐 일이 거의 없는 희소 특성들을 묶어 하나의 가상 특성으로 처리.

7.3.2 주요 하이퍼파라미터 상세 표

파라미터	의미	권장 범위	튜닝 우선순위
learning_rate	한 단계 학습량	0.01 ~ 0.1	1
n_estimators	트리 개수	200 ~ 2000+	1
num_leaves	한 트리의 최대 잎 수 (LightGBM 특유)	15 ~ 255	1
max_depth	트리 깊이 (-1 = 제한 없음)	-1 또는 3 ~ 12	2
min_data_in_leaf	잎 노드 최소 샘플 수 (과적합 억제)	10 ~ 100	2
feature_fraction	각 트리 특성 비율 (= colsample_bytree)	0.7 ~ 1.0	3
bagging_fraction	각 단계 샘플 비율 (=subsample)	0.7 ~ 1.0	3
bagging_freq	subsample 간격 (예 — 5)	1 ~ 10	3
lambda_l1 / lambda_l2	L1/L2 정규화	0 ~ 5	4
is_unbalance / scale_pos_weight	불균형 대응	True 또는 neg/pos	1 (불균형 시)

early_stopping_rounds	검증 점수 정체 시 중단	20 ~ 100	2
categorical_feature	범주형 특성 인덱스 직접 지정	list 또는 'auto'	데이터에 따라

num_leaves / max_depth 가이드

- num_leaves 와 max_depth 의 관계 - 대략 $\text{num_leaves} \leq 2^{\text{max_depth}}$ 가 권장. 예 - max_depth=6 이면 num_leaves 는 31~63 사이로.
- num_leaves 가 클수록 더 정밀하지만 과적합 위험이 커진다. 작은 데이터에서는 31 이하로.
- "학습은 빠르는데 결과가 이상하다" 면 num_leaves 를 줄이고 min_data_in_leaf 를 늘려 보자.

7.3.3 실전 예제 ① — 기본 이진 분류 (Titanic) [4 단계]

```
# 사전 설치: pip install lightgbm
# 데이터셋은 titanic 으로 한다.
from lightgbm import LGBMClassifier

lgbm = LGBMClassifier(
    n_estimators=500, learning_rate=0.05, max_depth=-1,
    num_leaves=31, subsample=0.8, colsample_bytree=0.8,
    random_state=42, n_jobs=-1)
lgbm.fit(X_tr, y_tr)
print("LightGBM 정확도:", lgbm.score(X_te, y_te))
```

7.3.4 실전 예제 ② — 검증 콜백 + early_stopping + 학습곡선

```
import lightgbm as lgb
from lightgbm import LGBMClassifier, early_stopping, log_evaluation
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X_tr, y_tr, test_size=0.2, stratify=y_tr, random_state=42)

lgbm = LGBMClassifier(
    n_estimators=2000,
    learning_rate=0.03,
    num_leaves=31,
    min_data_in_leaf=20,
    feature_fraction=0.9, bagging_fraction=0.85, bagging_freq=5,
    lambda_l2=1.0,
    random_state=42, n_jobs=-1)

lgbm.fit(X_train, y_train,
        eval_set=[(X_val, y_val)],
        eval_metric="auc",
        callbacks=[
            early_stopping(stopping_rounds=80),
            log_evaluation(period=100),
        ])

print("최적 라운드:", lgbm.best_iteration_)

# 학습 곡선 그리기
import matplotlib.pyplot as plt
```

```

hist = lgbm.evals_result_
plt.plot(hist["training"]["auc"], label="train")
plt.plot(hist["valid_1"]["auc"], label="valid")
plt.axvline(lgbm.best_iteration_, color="red", ls="--",
            label=f"best={lgbm.best_iteration_}")
plt.xlabel("boosting round"); plt.ylabel("AUC")
plt.legend(); plt.grid(alpha=0.3); plt.show()

```

LightGBM 의 callback 시스템은 학습 중 매 라운드 정보를 풍부하게 남긴다. `log_evaluation` (`period=100`) 으로 100 라운드마다 검증 점수를 출력하고, `early_stopping`(`stopping_rounds=80`) 으로 80 라운드 무개선 시 자동 중단. `evals_result_` 에 누적된 점수로 학습곡선을 그릴 수 있다.

7.3.5 실전 예제 ③ — 불균형 데이터 + 범주형 특성 직접 처리

```

import pandas as pd
from lightgbm import LGBMClassifier

# Titanic 의 인위적 불균형 버전 (양성 30%만 남김)
keep_pos = y_tr[y_tr == 1].sample(frac=0.3, random_state=42).index
keep_neg = y_tr[y_tr == 0].index
keep_idx = keep_pos.union(keep_neg)
X_imb = X_tr.loc[keep_idx].copy()
y_imb = y_tr.loc[keep_idx]

# 범주형 특성을 명시적으로 'category' 로 변환하면 LightGBM 이 자동 처리
for col in ["sex_male", "embarked_Q", "embarked_S", "pclass"]:
    if col in X_imb.columns:
        X_imb[col] = X_imb[col].astype("category")

neg, pos = (y_imb == 0).sum(), (y_imb == 1).sum()

lgbm = LGBMClassifier(
    n_estimators=600, learning_rate=0.05, num_leaves=31,
    scale_pos_weight=neg/pos,          # 불균형 대응
    random_state=42, n_jobs=-1)
lgbm.fit(X_imb, y_imb, categorical_feature="auto")

X_te2 = X_te.copy()
for col in ["sex_male", "embarked_Q", "embarked_S", "pclass"]:
    if col in X_te2.columns:
        X_te2[col] = X_te2[col].astype("category")

print("불균형 + 범주형 처리 LGBM 정확도:", lgbm.score(X_te2, y_te))

```

LightGBM 의 `categorical_feature="auto"` 옵션은 pandas 의 `category` dtype 을 자동 감지한다. one-hot 인코딩 없이도 "이 특성은 범주" 라는 정보를 알려 줄 수 있다. `scale_pos_weight` 와 함께 쓰면 불균형 데이터에서 안정적인 성능을 얻을 수 있다.

7.3.6 LightGBM 흔한 함정과 팁

LightGBM – 흔한 함정과 팁

- `num_leaves` 를 너무 크게 잡지 말 것. `max_depth=6` 이면 `num_leaves ≤ 64` 가 안전.

- 작은 데이터(< 수천 행)에서는 LightGBM 의 leaf-wise 가 단점이 된다 - 차라리 XGBoost 가 안정적.
- min_data_in_leaf 의 기본값 20 이 작은 데이터에는 너무 크다. 5 정도로 줄여 본다.
- "학습 중 진행 표시" 가 필요하면 callbacks=[log_evaluation(period=N)] 을 명시. 기본은 무음.
- categorical_feature 옵션을 쓰면 one-hot 보다 빠르고 메모리도 적게 든다. 단, pandas category dtype 으로 변환 필요.
- 윈도우에서 설치가 가끔 까다롭다 - pip install lightgbm 으로 안 되면 conda 권장.

7.4 CatBoost - 범주형 자동 처리와 Ordered Boosting

7.4.1 알고리즘 핵심 원리

CatBoost(Yandex, 2017) 의 차별점은 두 가지다. 첫째, 범주형 자동 처리 - "Ordered Target Statistics" 라는 영리한 인코딩으로 사용자가 직접 one-hot 이나 target encoding 을 하지 않아도 된다. 둘째, Ordered Boosting - 일반 그라디언트 부스팅이 빠지기 쉬운 "예측 편향"(prediction shift) 을 방지하는 학습 순서를 도입했다. 둘 다 "미래 정보 누설" 을 막는 장치다.



[그림 7-3] CatBoost 의 순서형 목표 인코딩 - 과거 데이터만으로 범주를 인코딩해 미래 정보 누설을 막는다.

CatBoost 가 다른 점

- Ordered Target Encoding - 시간 순서대로 데이터를 보며, 각 시점에서는 "그 이전" 데이터만으로 범주의 평균 목표값을 계산. 미래의 y 가 인코딩에 새지 않는다.
- Ordered Boosting - 일반 부스팅에서 같은 데이터로 잔차 계산과 트리 학습을 동시에 해서 생기는 예측 편향을 "여러 순열" 을 따로 두어 방지.
- 기본값이 매우 잘 잡혀 있어 튜닝을 거의 안 해도 무난한 성능이 나온다.

7.4.2 주요 하이퍼파라미터

파라미터	의미	권장 범위
iterations	트리 개수 (= n_estimators)	500 ~ 3000
learning_rate	한 단계 학습량	0.01 ~ 0.1
depth	트리 깊이 (균형 트리)	4 ~ 10
l2_leaf_reg	잎 L2 정규화	1 ~ 10

random_strength	분할 점수에 노이즈 추가	0 ~ 10
bagging_temperature	Bayesian 부트스트랩 강도	0 ~ 10
cat_features	범주형 특성 인덱스(필수)	list of int/str
loss_function	Logloss / MultiClass / RM SE 등	데이터에 맞춰
auto_class_weights	불균형 자동 가중치	'Balanced'
early_stopping_rounds	검증 정제 시 중단	30 ~ 100

7.4.3 실전 예제 ① — 범주형을 그대로 던지기 (Titanic)

```
# 사전 설치: pip install catboost
from catboost import CatBoostClassifier
import seaborn as sns, pandas as pd
from sklearn.model_selection import train_test_split

df = sns.load_dataset("titanic")
df = df[["survived", "pclass", "sex", "age", "sibsp", "parch", "fare", "embarked"]].copy()
df["age"] = df.groupby(["sex", "pclass"])["age"].transform(lambda s: s.fillna(s.median()))
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])

y = df["survived"]
X = df.drop(columns=["survived"])

# 범주형 특성은 인코딩 없이 그대로 - CatBoost 에게 위치만 알려 준다
cat_cols = ["pclass", "sex", "embarked"]

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

cb = CatBoostClassifier(
    iterations=600,
    learning_rate=0.05,
    depth=6,
    l2_leaf_reg=3.0,
    cat_features=cat_cols,          # 범주형 위치 지정
    eval_metric="AUC",
    random_seed=42,
    verbose=False)

cb.fit(X_tr, y_tr, eval_set=(X_te, y_te), early_stopping_rounds=50)

print("CatBoost 정확도:", cb.score(X_te, y_te))
print("최적 트리 수 :", cb.get_best_iteration())
```

CatBoost 는 one-hot 도, target encoding 도 안 해 준 채로 그냥 넣어도 된다 — cat_features 만 정확히 알려 주면 끝이다. 이것이 "전처리 코드가 짧아진다" 는 CatBoost 의 가장 큰 매력 중 하나다.

7.4.4 실전 예제 ② — 불균형 데이터 + auto_class_weights

```
from catboost import CatBoostClassifier

# 인위적 불균형 (양성 30%만 유지)
keep_pos = y_tr[y_tr == 1].sample(frac=0.3, random_state=42).index
keep_neg = y_tr[y_tr == 0].index
X_imb = X_tr.loc[keep_pos.union(keep_neg)]
y_imb = y_tr.loc[keep_pos.union(keep_neg)]

cb = CatBoostClassifier(
    iterations=500, learning_rate=0.05, depth=6,
    cat_features=["pclass", "sex", "embarked"],
    auto_class_weights="Balanced", # 클래스 빈도 자동 보정
    eval_metric="F1",
    random_seed=42, verbose=False)

cb.fit(X_imb, y_imb)
print("불균형 CatBoost F1:", cb.score(X_te, y_te))
```

7.4.5 CatBoost 함정/팁

CatBoost – 함정/팁

- cat_features 를 빠뜨리면 일반 그래디언트 부스팅과 다를 게 없어진다. "범주형 자동 처리" 가 핵심.
- 학습이 다른 라이브러리보다 느린 편 – 단, 튜닝을 거의 안 해도 되는 점이 보상.
- verbose=False 로 학습 로그 숨길 수 있다. 디폴트는 매 라운드 출력.
- 모델 저장 – cb.save_model("model.cbm"). 로딩 – CatBoostClassifier().load_model("model.cbm").

7.5 4대 라이브러리 한눈 비교표

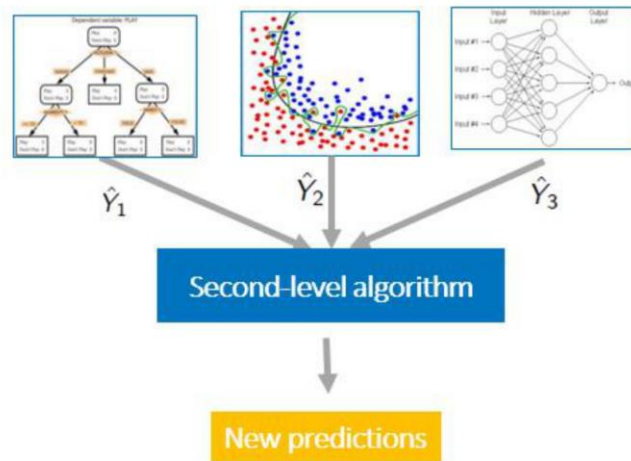
축	sklearn GBM	XGBoost	LightGBM	CatBoost
릴리스	2007~	2014	2016	2017
속도	느림	빠름	가장 빠름	보통
트리 성장	depth-wise	level-wise (균형)	leaf-wise (불균형)	depth-wise (균형)
범주형 처리	직접 인코딩	직접 인코딩 (1.5 부터 enable_categorical)	category dtype 자동	cat_features 만 지정
결측치 처리	직접 처리	자동	자동	자동
GPU 지원	없음	tree_method='gpu_hist'	device='gpu'	task_type='GPU'
주요 정규화	subsample	L1/L2 + γ + early_stop	L1/L2 + min_data_in_leaf	L2 + random_strength + Ordered Boosting

설치 난이도	사이킷런과 함께 자동	pip 한 줄, 안정적	pip 한 줄, 윈도우는 가끔 까다	pip 한 줄, 무난
기본값 품질	보통	보통 (튜닝 필요)	보통 (num_leaves 주의)	매우 좋음
권장 데이터 크기	수천 ~ 수만	수만 ~ 수백만	수십만 ~ 수천만	수천 ~ 수백만
적합한 상황	학습 전용	정확도 안정 추구	큰 데이터, 빠른 반복	범주형 많고 튜닝 시간 부족

7.6 참고 – 스택킹(Stacking) 짧게

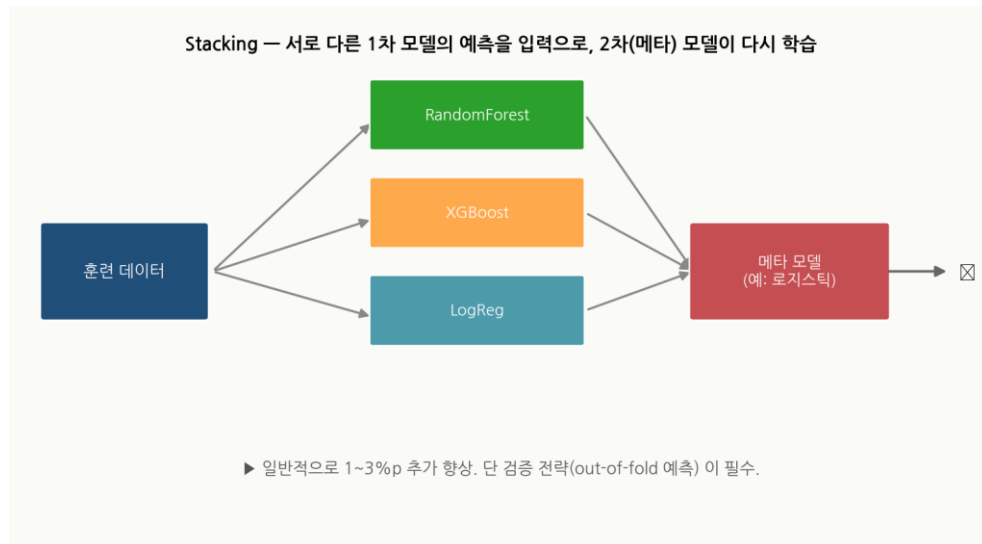
스택킹은 "서로 다른 1 차 모델의 예측을 입력으로 받아, 2 차(메타) 모델이 다시 학습하는" 두 단계 앙상블이다. 캐글 우승작에 자주 등장한다.

- 스택킹(Stacking); Meta Ensembling
- 스택킹은 여러 가지 알고리즘에서 예측한 값을 입력값으로(input) 하여 그 위에 추가적인 모형(주로 로지스틱 회귀분석)을 쌓는다(stacking).



source: <https://blogs.sas.com/content/subconsciousmusings/files/2017/05/modelstacking.png>

[그림 7-4] 스택킹(Stacking) — 1 차 모델 예측을 메타 모델이 입력으로 받아 다시 학습. — 출처: 강의자료 p.73.



[그림 7-5] 스택킹 — 1차 모델들의 예측을 메타 모델이 입력으로 받아 다시 학습.

스택킹 한 줄 정리

- 사이킷런에 StackingClassifier, StackingRegressor 가 내장돼 있다.
- 검증 전략(cv) 이 매우 중요 — 1차 모델의 train 예측이 학습된 데이터에 누설되면 안 된다. 보통 out-of-fold(OOF) 예측을 메타 모델 입력으로 쓴다.
- 추가 향상은 보통 1~3%p. 시간·복잡도 대비 이득이 항상 크지는 않다.

8 장. 모델 비교와 선택 가이드

여기까지 다섯 가지 앙상블(랜덤포레스트·AdaBoost·GBM·XGBoost·LightGBM) 을 다뤘다. 실무에서는 "어느 것을 골라야 하는가" 가 가장 자주 받는 질문이다. 이 장은 그 질문에 답하기 위한 비교 코드와 의사결정 흐름을 정리한다.

8.1 한 데이터에서의 한 줄 비교 코드 [5 단계]

```

from sklearn.ensemble import RandomForestClassifier

models = {
    "RandomForest": RandomForestClassifier(
        n_estimators=300, n_jobs=-1, random_state=42),
    "AdaBoost":      AdaBoostClassifier(n_estimators=200, random_state=42),
    "GBM":           GradientBoostingClassifier(
        n_estimators=300, learning_rate=0.05, max_depth=3, random_state=42),
    "XGBoost":       xgb,
    "LightGBM":      lgbm,
}

for name, m in models.items():
    if name in ("RandomForest", "AdaBoost", "GBM"):
        m.fit(X_tr, y_tr)
    p = m.predict(X_te); proba = m.predict_proba(X_te)[: , 1]
    print(f"{name:13s} acc={accuracy_score(y_te,p):.4f} "
          f"f1={f1_score(y_te,p):.4f} auc={roc_auc_score(y_te,proba):.4f}")
  
```

Titanic 같은 작은 데이터에서는 모델 간 점수 차이가 1~3%p 정도로 미세하다. 데이터가 클수록 XGBoost·LightGBM 의 우위가 커지며, 범주형 변수가 많을수록 CatBoost 의 우위가 두드러진다.

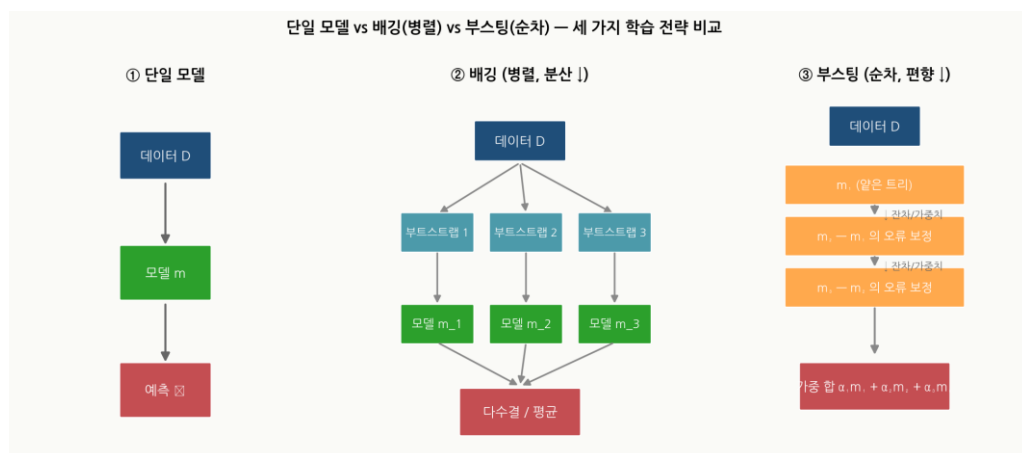
8.2 학습 곡선 — learning_rate × n_estimators [6 단계]

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
for lr in [0.01, 0.05, 0.1, 0.3]:
    m = GradientBoostingClassifier(
        n_estimators=400, learning_rate=lr, max_depth=3,
        random_state=42).fit(X_tr, y_tr)
    test_scores = [
        accuracy_score(y_te, y) for y in m.staged_predict(X_te)]
    plt.plot(test_scores, label=f"lr={lr}")

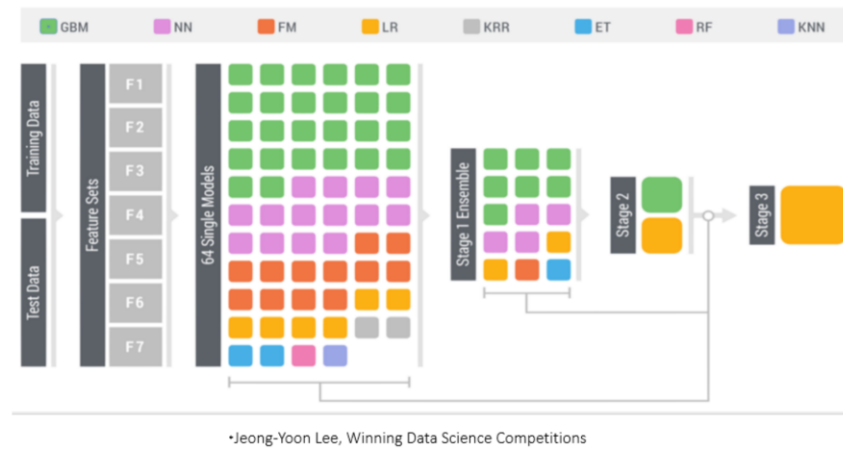
plt.xlabel("부스팅 단계 (n_estimators)")
plt.ylabel("테스트 정확도")
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

8.3 단일 vs 배깅 vs 부스팅 한눈 도식



[그림 8-1] 단일 모델·배깅·부스팅 — 세 가지 학습 전략 비교.

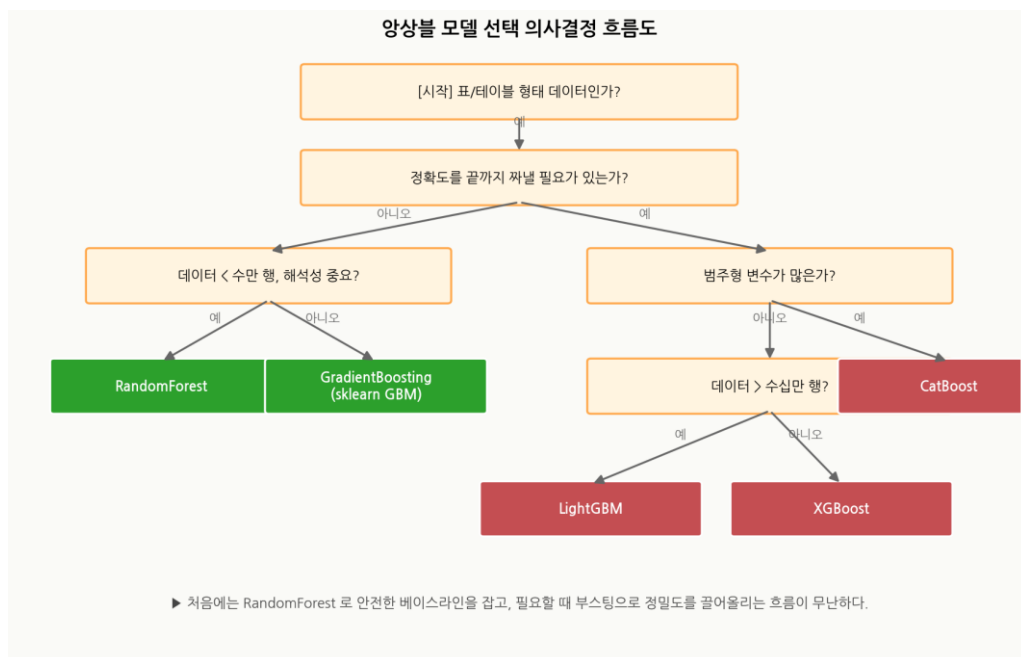
- 스택킹(Stacking); Meta Ensembling
- 필요한 연산량이 훨씬 증가하지만 정확도는 상대적으로 소량 증가한다.
-> 실무보다는 Kaggle과 같은 대회에서 자주 사용된다.



source: <https://blogs.sas.com/content/subconsciousmusings/files/2017/05/stackedapproach.png>

[그림 8-2] 스택킹(Stacking) — 1·2 차 모델의 학습 흐름. — 출처: 강의자료 p.74.

8.4 모델 선택 의사결정 흐름도



[그림 8-3] 앙상블 모델 선택 의사결정 흐름도.

8.5 데이터 특성별 권장 모델 표

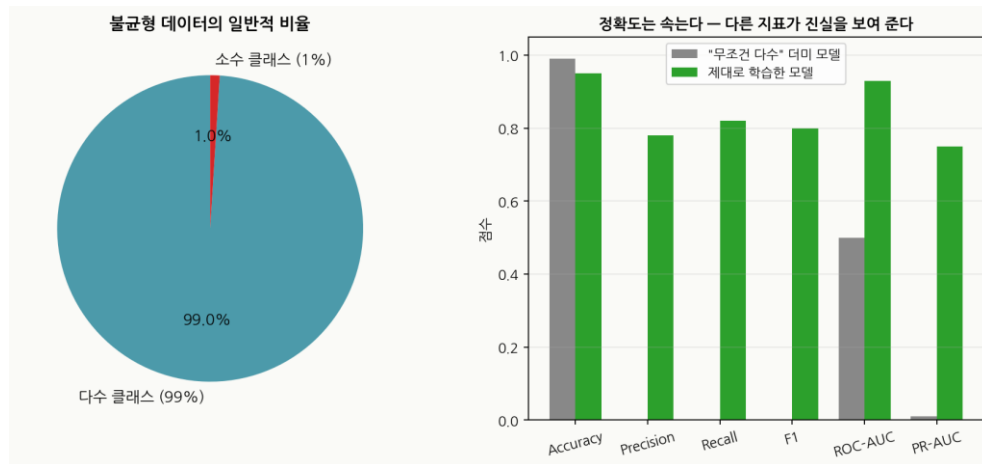
상황	권장 1 순위	이유
수만 행 이내, 베이스라인 필요	RandomForest	튜닝 거의 안 해도 안전. 해석성도 무난.
수십만~수백만 행, 정밀도 추구	LightGBM 또는 XGBoost	히스토그램 기반으로 빠르고 정확.

범주형 변수 5 개 이상	CatBoost	Ordered Target Encoding 으로 전처리 부담 ↓.
클래스 불균형 (1:10 이상)	XGBoost (scale_pos_weight)	정규화·early_stopping 으로 안정.
회귀 + 이상치 많음	GBM (Huber loss) 또는 XGBoost (huber)	Huber 손실로 견고.
분위수 예측 (예 — 신뢰구간)	GBM (quantile)	사이킷런 GBM 의 quantile loss.
대회·실무 1 등 시도	XGBoost + LightGBM + CatBoost 앙상블 (스택킹)	보통 1~3%p 추가 향상.
해석성이 매우 중요	RandomForest + permutation importance	트리 평균은 직관적, 순열은 객관적.

9 장. 클래스 불균형과 실전 함정

2 교시 전처리 교재에서 보았듯이 클래스 불균형(예 — 사기 0.1%, 정상 99.9%)에서는 정확도가 의미 없고, 단순 학습은 "무조건 다수 클래스" 라는 게으른 답을 내놓는다. 부스팅 모델에는 이를 해결하는 기본 도구가 갖춰져 있다.

9.1 클래스 불균형 문제



[그림 9-1] 클래스 불균형 — 정확도는 속는다. 다른 지표가 진실을 보여 준다.

9.1.1 다수 클래스 답의 함정

사기 탐지 데이터에서 "모두 정상" 이라고 답하면 정확도는 99.9% 다. 하지만 사기 한 건도 못 잡는다. 정확도는 다수 클래스에 끌려가는 지표라 의미가 없다.

9.1.2 평가지표 변경

불균형 데이터에서는 다음 지표를 함께 본다.

- Precision — 양성 예측 중 진짜 양성 비율. "잘못 알람" 비용을 본다.
- Recall (Sensitivity) — 진짜 양성 중 맞춘 비율. "놓친 양성" 비용을 본다.

- F1 — Precision 과 Recall 의 조화평균. 두 지표의 균형을 본다.
- ROC-AUC — 분류 임계값을 다 훑은 전체 능력. 클래스 분포에 영향이 비교적 작다.
- PR-AUC — Precision-Recall 곡선 아래 면적. 매우 불균형한 경우 ROC-AUC 보다 민감.

9.2 모델 수준 — 가중치 부여

- 사이킷런 — `class_weight="balanced"` — 클래스 빈도에 반비례하게 자동 가중.
- XGBoost — `scale_pos_weight = 음성 샘플 수 / 양성 샘플 수`.
- LightGBM — `is_unbalance=True` 또는 `scale_pos_weight` 직접 지정.
- CatBoost — `auto_class_weights="Balanced"`.

9.3 데이터 수준 — 리샘플링

리샘플링 기법

└─ 오버샘플링

| └─ 단순 복제(Random Oversampling) ← 과적합 위험

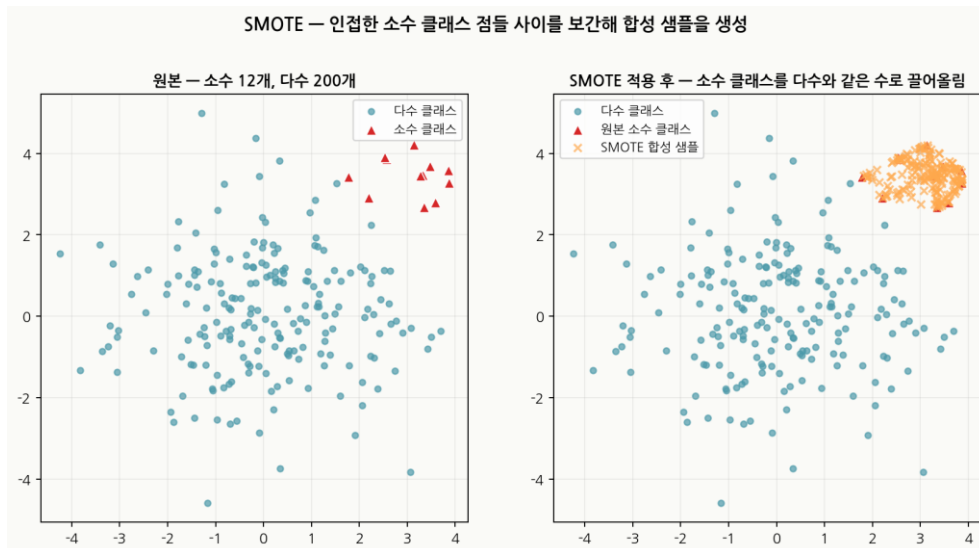
| └─ SMOTE: 소수 클래스 점들 사이에 합성 샘플 생성 ← 권장

└─ 언더샘플링

| └─ Random Undersampling ← 정보 손실 위험

└─ 혼합 (SMOTE + Tomek links 등)

imbalanced-learn(imblearn) 라이브러리가 SMOTE 등 거의 모든 기법을 사이킷런 호환 API 로 제공한다.



[그림 9-2] SMOTE — 인접한 소수 클래스 점들 사이를 보간해 합성 샘플을 생성.

9.4 코드 실습 — 불균형 처리 두 가지

9.4.1 [7 단계] `scale_pos_weight` (모델 수준 가중치)

인위적 불균형 데이터 만들기 (Titanic 의 30 % 만 양성으로)

```
keep_pos = y_tr[y_tr == 1].sample(frac=0.3, random_state=42).index
keep_neg = y_tr[y_tr == 0].index
```

```

keep_idx = keep_pos.union(keep_neg)
X_imb, y_imb = X_tr.loc[keep_idx], y_tr.loc[keep_idx]

pos = (y_imb == 1).sum(); neg = (y_imb == 0).sum()

# (a) 가중치 없이
a = XGBClassifier(eval_metric="logloss", random_state=42, n_jobs=-1)
a.fit(X_imb, y_imb)

# (b) scale_pos_weight 사용
b = XGBClassifier(
    scale_pos_weight=neg / pos,
    eval_metric="logloss", random_state=42, n_jobs=-1)
b.fit(X_imb, y_imb)

for name, m in [("균형 가중치 X", a), ("균형 가중치 0", b)]:
    p = m.predict(X_te)
    print(f"{name} recall={f1_score(y_te,p):.4f} "
          f"auc={roc_auc_score(y_te, m.predict_proba(X_te)[: , 1]):.4f}")

```

scale_pos_weight 를 켜면 보통 Recall 이 크게 올라가지만 Precision 은 살짝 떨어진다.
F1·AUC 로 종합 평가해야 한다.

9.4.2 [8 단계] SMOTE — 단계별 명시 호출 (Pipeline 미사용)

Pipeline 을 쓰지 않고 각 단계를 명시적으로 호출하는 형태로 다시 쓴다.

```

# 사전 설치: pip install imbalanced-learn
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier
from sklearn.metrics import f1_score, roc_auc_score

# 1 단계: SMOTE 로 학습 데이터만 합성 (테스트 데이터는 절대 건드리지 않음!)
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X_imb, y_imb)

print(f"SMOTE 전 양성: {(y_imb==1).sum()} / 음성: {(y_imb==0).sum()}")
print(f"SMOTE 후 양성: {(y_res==1).sum()} / 음성: {(y_res==0).sum()}")

# 2 단계: 모델 학습 (이제부터는 일반 분류와 동일)
clf = XGBClassifier(eval_metric="logloss", random_state=42, n_jobs=-1)
clf.fit(X_res, y_res)

# 3 단계: 테스트 평가
y_pred = clf.predict(X_te)
y_proba = clf.predict_proba(X_te)[: , 1]

print("SMOTE + XGB F1 :", f1_score(y_te, y_pred))
print("SMOTE + XGB AUC:", roc_auc_score(y_te, y_proba))

```

SMOTE 사용 시 절대 잊지 말 것

- SMOTE 는 "학습 데이터에만" 적용한다. 테스트 데이터에는 절대 적용하지 않는다 – 그렇지 않으면 평가가 부풀려진다.
- scale_pos_weight 와 SMOTE 를 동시에 쓰면 양성 클래스가 "이중으로" 증폭되어 오히려 망친다 – 둘 중 하나만 고른다.

- 교차검증 시에는 폴드 분리 후 "각 폴드 내에서" SMOTE 를 다시 적용해야 한다. 분리 전에 SMOTE 를 하면 합성 샘플이 검증 폴드에 새어 들어간다.

9.5 부스팅 튜닝 체크리스트

튜닝 순서

- ① 검증 세트를 먼저 분리한다 (train_test_split 한 번 더).
- ② early_stopping_rounds 를 켜고 n_estimators 를 크게(1000~3000) 잡는다.
- ③ 먼저 learning_rate=0.05, max_depth=5 (XGB) 또는 num_leaves=31 (LGBM) 으로 베이스라인.
- ④ 그 다음 max_depth/num_leaves 와 min_data_in_leaf 로 정규화 강도를 조정.
- ⑤ subsample / colsample_bytree 로 추가 정규화.
- ⑥ 마지막으로 reg_lambda / reg_alpha 같은 L2/L1 정규화.
- ⑦ 그래도 안 되면 데이터 자체를 의심 - 누수, 잘못된 평가지표, 작은 검증 세트.

9.6 9장 함정 모음

실전 함정 모음

- 데이터 누수(data leakage) - 미래 정보가 학습 데이터에 새는 경우. 시계열에서 random split 을 하면 발생.
- 잘못된 평가지표 선택 - 불균형 데이터에 정확도를 쓰는 등.
- OOB 와 테스트 점수 차이가 크면 의심 - 그룹 누수, 시계열 누수.
- scale_pos_weight 와 SMOTE 동시 적용 금지.
- learning_rate 가 너무 작으면(예 - 0.001) n_estimators 가 폭증해 학습 시간이 비현실적이 된다.
- 검증 세트가 너무 작으면 early_stopping 이 우연에 휘둘린다.
- 범주형 카디널리티가 매우 높으면(예 - 우편번호 10 만 개) target encoding 의 부작용에 주의 - CatBoost 의 Ordered Encoding 권장.

10 장. 단원 정리·통합 과제·체크리스트

10.1 한 페이지 정리

축	랜덤포레스트 (배경)	부스팅 (XGB/LGBM/Cat)
학습 방식	병렬	순차
줄여주는 것	분산	편향
기본 학습기	깊은 트리	얕은 트리 (depth 3~8)
다양성 원천	부트스트랩 + 특성 무작위 선택	잔차 학습 + subsample
과적합 위험	낮음 (n_estimators 늘려도 안전)	중간 (튜닝 필수)
핵심 파라미터	n_estimators, max_features	learning_rate, n_estimators, max_depth
병렬화	쉬움	트리 내부만

불균형 대응	class_weight="balanced"	scale_pos_weight / SMOTE
언제 쓰나	안전한 베이스라인	정밀도 끝까지 짜내기
사용 빈도(실무)	★★★★☆	★★★★★ (테이블 데이터의 표준)

10.2 자가 점검 퀴즈 (10 문항)

- Q1. 앙상블이 분산을 $1/N$ 로 줄이려면 어떤 조건이 필요한가?
- Q2. 부트스트랩 표본에서 약 36.8% 가 OOB 로 남는 이유를 식으로 설명하라.

10.3 통합 과제

10.3.1 [기본] 모델 비교 표

- Titanic 데이터에서 단일 결정트리·로지스틱 회귀
·RandomForest·GBM·XGBoost·LightGBM·CatBoost 일곱 모델의 정확도·F1·AUC·학습 시간을 비교 표로 작성.
- 어떤 모델이 어떤 지표에서 우위인지, 그 이유를 한 단락으로 적는다.

10.3.2 [심화] XGBoost $lr \times n_estimators$ 16 칸 히트맵

- $learning_rate \in [0.01, 0.05, 0.1, 0.3] \times n_estimators \in [100, 300, 500, 1000]$ 의 16 개 조합으로 XGBoost 학습.
- 각 조합의 테스트 AUC 를 16 칸짜리 표(또는 히트맵) 로 그린다.
- 관찰한 패턴(작은 $lr \times$ 큰 N 이 안정적인 영역) 을 한 단락으로 정리.

10.3.3 [심화] 불균형 데이터 3 방법 비교 보고서

- Titanic 의 인위 불균형 버전(양성 30%) 에서 (a) 가중치 없음, (b) scale_pos_weight, (c) SMOTE 세 방법으로 학습.
- 각각의 재현율·정밀도·F1·AUC 를 표로 비교.
- 혼동행렬 시각화를 포함해 "각 방법이 무엇을 양보하고 무엇을 얻었는가" 를 분석.

10.3.5 [선택] 특성 중요도 비교

- Titanic 데이터에서 feature_importances_ 와 permutation_importance 결과를 같은 그림(나란히 막대그래프) 에 비교 시각화.
- 두 결과가 다른 특성이 있다면 어떤 이유로 다를지 추측해 한 줄 적는다.